

Literature Survey

Paper Title	Method	Dataset	Advantages	Limitations	Research Gap
IaC Generation with LLMs: An Error Taxonomy and A Study on Configuration Knowledge Injection	Evaluates LLM-based Terraform code generation across Naive RAG, Graph RAG, and three graph enhancements (GR-OptMatch, GR-LLMSum, GR-Ref) using cloud emulation (LocalStack) and automated validation.	IaC-Eval benchmark enhanced with LocalStack, comprising 458 Terraform generation prompts and official AWS Provider v5.90.0 documentation.	Boosts technical validation success from 37.2% to 84.2% and overall success from 27.1% to 62.6%. Fully local execution via LocalStack eliminates AWS account costs, network latency, and accidental resource provisioning. Provides a detailed two-dimensional taxonomy of 19 error types.	Intent validation plateaus at 72%-76% due to the 'Correctness-Congruence Gap'. Graph RAG with full reference modeling (GR-Ref) causes cognitive overload and degrades performance on simpler prompts. Restricted exclusively to Terraform configurations for AWS.	Existing knowledge injection approaches focus primarily on single-provider (AWS) Terraform syntax, creating a gap in intent alignment ('Correctness-Congruence Gap') and failing to optimize context retrieval across heterogeneous multi-cloud frameworks.
GENSIAC: Toward Security-Aware Infrastructure-as-Code Generation with Large Language Models	Instruction fine-tuning of base LLMs (CodeLlama 7B/13B) using LoRA and 8-bit quantization to inject security weakness awareness into generation and inspection workflows.	GENSIAC Dataset: 22K data points (11K code generation, 11K code inspection pairs) covering Ansible, Chef, Puppet across 1,413 repos, plus 148 test scripts (229 security flaws).	Significantly raises security weakness detection F1-score from ~0.30 to 0.858 (outperforming GPT-4). Improves syntactical and functional correctness scores to 1.000 and 0.979. Resource-efficient fine-tuning requiring under 10 hours per epoch on an A100 GPU.	Does not cover all security weakness categories (e.g., CWE-276) due to reliance on upstream static scanning tools. Cross-language transfer performance is lower when evaluated on Ansible compared to Chef or Puppet. Only tested using LoRA without exploring alternative PEFT strategies or runtime cloud validation.	Current security-aware fine-tuning depends heavily on static scanning tools resulting in incomplete flaw coverage, inconsistent cross-language transfer, and an absence of automated runtime validation in cloud environments.
INFRAFIX: Technology-Agnostic Repair of Infrastructure as Code	A polyglot Automated Program Repair (APR) framework utilizing an extended object-oriented Intermediate Representation (IR), a normalization database, system-call-based state inference, and an SMT-based repair engine.	Curated Polyglot Dataset: 12,358 Ansible, 4,868 Puppet, 3,568 Chef, 1,300 Terraform, and 13 Tortoise scripts, yielding 254,288 repair scenarios.	Technology-agnostic architecture capable of operating across Ansible, Puppet, Chef, and Terraform simultaneously. High overall repair success rate of 95.7% across 254,288 scenarios. Includes an automated mutation-based repair	Fails when resource names cannot be statically resolved into strings or are defined in uninstantiated classes/functions. Struggles with undefined variable references, shared variable side-effects, and array-based title definitions. Requires implementing	Symbolic and constraint-driven repair engines cannot dynamically interpret runtime variables, non-filesystem cloud resources, or complex multi-resource dependencies without manual specification, leaving a gap in

Paper Title	Method	Dataset	Advantages	Limitations	Research Gap
			scenario generator for large-scale evaluation.	separate state inference modules for non-filesystem resources.	handling dynamically resolved variables, complex security reasoning, and active cloud runtime feedback.
GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation	End-to-end framework pairing GenKubeDetect (CodeT5+ fine-tuned with LoRA for multi-label classification) with GenKubeResolve (Mistral-7B guided by few-shot prompts for localization, reasoning, and remediation).	Unified Misconfiguration Index (UMI) with 169 misconfiguration classes plus 276,520 Kubernetes Configuration Files (KCFs) from 'The Stack' and 700MB of domain text.	High detection precision (0.990) and superior recall (0.999) compared to static rule-based tools. Provides 100% expert-validated line-level localization, reasoning, and actionable remediation. Fast adaptation to new misconfigurations using as few as 1-5 labeled examples.	Constrained by CodeT5+'s 512-token input limit, leaving 17% of longer deployment files unanalyzed. High false-positive rates during few-shot adaptation for highly similar misconfiguration categories (e.g., RBAC rules). Qualitative evaluations were limited to a small sample of false positives and resolution outputs.	Specialized misconfiguration detection models suffer from restrictive context token limits on long deployment files and lack multi-agent integration capable of unifying Kubernetes manifests with broader cloud infrastructure services (e.g., AWS EC2, S3, IAM).
Practical Integration of Large Language Models into Enterprise CI/CD Pipelines for Security Policy Validation: An Industry-Focused Evaluation	Enterprise DevSecOps framework augmenting static scanners (OPA/Checkov) with an LLM Ensemble (GPT-4, domain fine-tuned models, and ReAct input sanitization filtering) inside Jenkins/Bamboo CI/CD pipelines.	Synthetic IaC Suite: 500 cases (300 K8s, 200 IAM/Terraform) plus real-world validation on 318 enterprise Helm charts (~47k LOC).	Ensemble method achieves high classification accuracy (F1 = 0.95) at low latency (3.1s). ReAct input filtering reduces prompt injection attack success from 60% down to 7%-12%. Reduces manual security review times by 60% and monthly rule maintenance workload by 70%.	Ensemble method requires 3x computational resources, increasing operational costs. Generates false security alarms (~23% hallucination error rate) and exhibits over-confidence on ambiguous code. Evaluation restricted to mid-scale environments and synthetic datasets.	In-pipeline LLM policy validation remains computationally expensive and prone to hallucinations (~23%), highlighting a gap for cost-optimized, multi-agent frameworks that integrate automated self-healing with real-time cloud environment testing